

Cognitive Outsourcing with Suspend-and-Inject Generation for Scalable Embodied Intelligence

GH.Qian

qgh132555@qq.com

github.com/MonSp/cognitive-outsourcing

Abstract

This paper presents a five-dimensional investigation of Cognitive Outsourcing (CO) with Suspend-and-Inject Generation (SIG)—a novel edge-AI architecture enabling lightweight on-device language models to dynamically access external cognitive resources while preserving continuous attention state. The original CO work demonstrated up to 96% prefill token savings and $1.57\times$ end-to-end speedups, but also revealed critical open questions about the theoretical foundations, practical limitations, and broader applicability of the paradigm.

We benchmark CO+SIG on Qwen3.5-0.8B and Qwen3.5-4B (Q4_K_M quantization) across nine multi-turn scenarios, confirming **73–97% prefill savings** and total end-to-end speedups of **$2.38\times$ (0.8B) and $2.70\times$ (4B)**. In autonomous tool-calling mode, where generation token counts are recorded, we find that per-token generation rates are nearly identical between AppLoop and SIG (within 2%), demonstrating that the apparent generation time differences are output-length-driven and the total-time speedup originates from prefill savings. KV-cache continuity through SIG enables the 0.8B model to improve from 0–5% to 68–100% tool accuracy on deep-chain scenarios, providing indirect evidence that KV-cache preservation is critical for small-model multi-turn agent capability.

Beyond the benchmarks, we present a structured five-dimensional analysis spanning information theory, cache lifecycle management, architectural compatibility, teacher-student optimization, and privacy guarantees. Each dimension combines a proposed theoretical framework with the first available empirical measurement where possible: **(R1)** a direct measurement of attention distribution shift between SIG injection and full re-encoding on Qwen2.5-0.5B (head agreement 0.25 early $\rightarrow$ 0.43 late, cosine similarity 0.725), confirming that early layers are most sensitive to injection; **(R2)** KV-cache degradation measurements across 6–10 injection rounds showing no degradation on weather recall (stable 0.50–1.00) for both models; **(R3)** a cross-architecture numpy simulation engine with empirical calibration from CO benchmarks; **(R4)** the first capability gap measurement (0.8B alone=0.20, +CoT=1.00, +SIG=0.79, teacher margin=0.72); and **(R5)** a PII redaction and intent-only outsourcing concept demo across four query types. The theoretical formalisms (upper bounds, degradation models, Pareto frontiers) remain conjectural and await rigorous proof and empirical parameter estimation, but each dimension now has at minimum a simulation framework or concept demonstration, and R1/R2/R4 have the first direct measurements of their core hypotheses.

The three empirical entry points (`co_benchmark.py`, `sig_benchmark.py`, `transformer_bench.py`) form a reusable testing infrastructure for future CO+SIG research.

1 Introduction

The first generation of Cognitive Outsourcing [1] established a compelling empirical case for Suspend-and-Inject Generation as an inference-engine primitive: by preserving the model’s key-value (KV) cache across external interactions, SIG eliminates the quadratic prefill overhead of traditional tool-calling loops and maintains cognitive continuity. The experimental

results—up to 96% prefill token savings, 87% prefill time reduction, $1.57\times$ end-to-end speedup on 0.8B models, and a $3\times$ improvement in long-context information coverage—demonstrated the architectural viability of the approach.

However, the original work also surfaced five fundamental open questions:

1. **Information-Theoretic Grounding:** What is the provable upper bound on information loss between SIG injection and full re-encoding? How does attention distribution shift across layers, and which layers are most sensitive to injection?
2. **Cache Lifecycle Management:** Does the KV cache inevitably degrade across multiple injection rounds? What is the shape of the degradation curve, and can cache eviction and compression strategies extend the effective horizon?
3. **Architectural Generality:** Is SIG fundamentally tied to the Transformer’s KV-cache mechanism, or can it be implemented on alternative architectures such as State Space Models (Mamba), RWKV, and xLSTM?
4. **Teacher-Student Optimization:** Is there an optimal capability gap between cloud teacher and edge student? Can chain-of-thought complexity be adaptively controlled to match student capacity?
5. **Privacy Guarantees:** Can the privacy claims of cognitive outsourcing—that sensitive data remains local—be formally quantified? What are the dominant leakage channels, and how effective are differential privacy defenses?

This paper addresses these questions through an integrated approach that combines empirical benchmarks with structured theoretical analysis. Section 2 extends the original CO+SIG benchmarks to the Qwen3.5 model family (0.8B and 4B, Q4_K_M quantization, NVIDIA RTX 4070 SUPER 12GB) across teacher-precomputed and autonomous tool-calling modes, confirming prefill savings and quantifying the origin of speedup through generation token count analysis. Section 3 presents the five-dimensional analysis, where each dimension is organized as a self-contained unit: theoretical framework, currently available empirical measurement (where one exists), and proposed directions for future investigation. Throughout the paper, we are explicit about what has been measured and what remains conjectural.

2 Extended CO+SIG Benchmarks

We conducted comprehensive benchmarks of the CO+SIG architecture using the Qwen3.5 model family on an NVIDIA GeForce RTX 4070 SUPER (12 GB). Two quantized models were tested: Qwen3.5-0.8B-Q4_K_M and Qwen3.5-4B-Q4_K_M. Each of nine benchmark scenarios was run 3 times per mode; all reported values are averages.

2.1 Experimental Setup

The benchmark suite comprises nine scenarios spanning short tool chains (2–4 calls), medium chains (9–12 calls), and deep chains (14–22 calls). The scenarios are: (1) Long-seq (22 turns), (2) Multi-tool chain, (3) Rapid-fire (12 turns), (4) Long-document + tools, (5) Mixed conversation, (6) Deep chain (14 tools), (7) Travel planning (11 tools), (8) Code debugging, and (9) Cross-reference (9 tools). Two operational modes were evaluated:

- **Teacher-precomputed mode:** A cloud teacher generates chain-of-thought (CoT) plans offline. The local model executes the plan either through traditional AppLoop (full re-prefill after each tool call) or SIG (KV-cache injection). Since plans are precomputed,

network variance is eliminated and pure inference performance is isolated. **Important limitation:** generation token counts were not recorded in this mode, so generation time comparisons between AppLoop and SIG are uninterpretable—the total end-to-end time speedup is the only valid efficiency metric.

- **Autonomous tool-calling mode:** The model generates its own tool calls without a teacher-provided plan. This is the more challenging setting—the model must maintain its reasoning chain across turns while autonomously deciding when and which tools to invoke. **This mode does record generation token counts**, enabling analysis of where time differences originate.

2.2 Teacher-Precomputed Mode Results

In this mode, the teacher-provided CoT plan ensures identical task completion across AppLoop and SIG, making efficiency comparison clean. However, as noted above, generation token counts were not recorded, so the breakdown of generation vs. prefill time contributions cannot be fully interpreted.

Table 1: CO+AppLoop vs. CO+SIG — 0.8B model (Qwen3.5-0.8B-Q4_K_M).

Scenario	AppLoop Gen(s)	SIG Gen(s)	AppLoop PF(s)	SIG PF(s)	AppLoop Total(s)	SIG Total(s)	Speedup
1 Long-seq (22 turns)	2.88	1.92	2.37	0.29	5.25	2.21	2.38×
2 Multi-tool chain	0.72	0.77	0.04	0.04	0.77	0.81	0.95×
3 Rapid-fire (12 turns)	2.16	1.23	0.68	0.17	2.84	1.40	2.03×
4 Long-document + tools	1.05	0.64	0.17	0.10	1.21	0.74	1.64×
5 Mixed conversation	2.63	1.53	0.33	0.10	2.96	1.64	1.80×
6 Deep chain (14 tools)	9.28	2.06	1.94	0.20	11.22	2.26	4.96×
7 Travel planning (11 tools)	3.21	2.21	0.89	0.18	4.10	2.39	1.72×
8 Code debugging	1.12	0.55	0.29	0.11	1.41	0.67	2.10×
9 Cross-reference (9 tools)	1.23	1.00	0.52	0.13	1.75	1.12	1.56×
Average	2.70	1.32	0.80	0.15	3.50	1.47	2.38×

Table 2: CO+AppLoop vs. CO+SIG — 4B model (Qwen3.5-4B-Q4_K_M).

Scenario	AppLoop Gen(s)	SIG Gen(s)	AppLoop PF(s)	SIG PF(s)	AppLoop Total(s)	SIG Total(s)	Speedup
1 Long-seq (22 turns)	8.03	3.26	6.97	0.54	15.00	3.80	3.95×
2 Multi-tool chain	0.54	0.54	0.10	0.09	0.63	0.64	0.98×
3 Rapid-fire (12 turns)	9.16	3.68	2.64	0.31	11.80	3.99	2.96×
4 Long-document + tools	5.91	3.19	0.85	0.17	6.75	3.36	2.01×
5 Mixed conversation	5.63	3.43	0.91	0.26	6.54	3.70	1.77×
6 Deep chain (14 tools)	12.34	2.80	4.69	0.45	17.03	3.24	5.26×
7 Travel planning (11 tools)	10.63	3.56	3.13	0.39	13.76	3.95	3.48×
8 Code debugging	2.70	1.89	0.94	0.30	3.64	2.19	1.66×
9 Cross-reference (9 tools)	4.21	4.94	1.73	0.28	5.94	5.22	1.14×
Average	6.57	3.03	2.44	0.31	9.01	3.34	2.70×

Table 3: Aggregate CO teacher-precomputed metrics.

Metric	0.8B AppLoop	0.8B SIG	4B AppLoop	4B SIG
Avg Generation Time	2.70s	1.32s	6.57s	3.03s
Avg Prefill Time	0.80s	0.15s	2.44s	0.31s
Avg Total Time	3.50s	1.47s	9.01s	3.34s
End-to-End Speedup		2.38×		2.70×
Tool Accuracy	100%	100%	100%	100%
GPU VRAM	~1.3 GB	~1.4 GB	~3.9 GB	~4.0 GB

Caveat on generation time comparisons: With generation token counts now recorded (see Table 6), we can directly analyze where time differences originate. For 0.8B: AppLoop avg 699 gen.toks at 274 tok/s, SIG avg 354 gen.toks at 281 tok/s. For 4B: AppLoop avg 625 gen.toks at 99 tok/s, SIG avg 296 gen.toks at 101 tok/s. **Per-token generation rates are nearly identical between AppLoop and SIG** (within 2% for both models). Generation time differences are therefore output-length-driven, not architectural. The total end-to-end speedup originates primarily from prefill savings (93% token reduction).

2.3 Autonomous Tool-Calling Mode Results

In autonomous mode, **generation token counts were recorded**, enabling a more honest analysis of where time is spent. This mode also introduces a fairness concern: AppLoop and SIG may complete different numbers of tool calls, make different decisions, and produce different-length outputs.

Table 4: Tool accuracy in autonomous mode — 0.8B model.

Scenario	AppLoop	SIG	Advantage
1 Long-seq (22 turns)	1/22 (5%)	15/22 (68%)	SIG +63pp
2 Multi-tool (4 tools)	0/4 (0%)	3/4 (75%)	SIG +75pp
3 Rapid-fire (12 turns)	0/12 (0%)	8/12 (67%)	SIG +67pp
4 Long-document (4 tools)	0/4 (0%)	4/4 (100%)	SIG +100pp
5 Mixed (4 tools)	4/4 (100%)	4/4 (100%)	Tie
6 Deep chain (14 tools)	0/14 (0%)	14/14 (100%)	SIG +100pp
7 Travel planning (11 tools)	0/0 (N/A)	7/13 (54%)	SIG +54pp
8 Code debugging (4 tools)	3/4 (75%)	2/2 (100%)	SIG +25pp
9 Cross-reference (9 tools)	0/0 (N/A)	5/11 (45%)	SIG +45pp

For the 0.8B model, AppLoop catastrophically fails on 6 of 9 scenarios (0% tool accuracy), effectively incapable of maintaining any tool-using agentic behavior over multiple turns. SIG rescues these scenarios, achieving 68–100% accuracy on five of the six previously failed scenarios. This provides indirect evidence consistent with the R1 hypothesis: KV-cache continuity appears essential for small-model multi-turn agent capability. However, this does not constitute a direct measurement of information loss—it is a behavioral outcome that is *consistent with* the information-theoretic framework, not a validation of it.

Table 5: Tool accuracy in autonomous mode — 4B model.

Scenario	AppLoop	SIG	Advantage
1 Long-seq (22 turns)	22/22 (100%)	22/22 (100%)	Tie
2–5 (Simple to medium)	100%	100%	Tie
6 Deep chain (14 tools)	14/14 (100%)	13/14 (93%)	AppLoop +7pp
7 Travel planning (11 tools)	4/7 (57%)	0/0 (N/A)	AppLoop
8 Code debugging	4/6 (67%)	2/2 (100%)	SIG +33pp
9 Cross-reference (9 tools)	8/8 (100%)	7/7 (100%)	Tie

The 4B model is generally capable in both modes. The SIG rollbacks in Travel planning (scenario 7) indicate instability in the autonomous tool-calling loop for this specific multi-city itinerary task.

Table 6: Generation token counts and timing in autonomous SIG mode — 4B model.

Scenario	AppLoop Gen(s)	SIG Gen(s)	AppLoop gen.toks	SIG gen.toks	AppLoop tok
1 Long-seq (22 turns)	7.63	7.04	690	690	90.4
2 Multi-tool (4 tools)	3.51	1.68	344	159	98.0
3 Rapid-fire (12 turns)	4.35	4.09	402	402	92.4
4 Long-document	2.71	2.80	255	280	94.1
5 Mixed	3.59	4.90	340	480	94.7
6 Deep chain (14 tools)	5.56	6.08	504	562	90.6

This table reveals the central finding about generation time. **In scenarios where generation token counts are equal (Long-seq: 690 vs. 690; Rapid-fire: 402 vs. 402), generation times are comparable (7.63s vs. 7.04s; 4.35s vs. 4.09s).** Per-token generation rates range from 90–100 tok/s across both modes, with variance attributable to CPU/GPU scheduling noise rather than systematic architectural differences. In scenarios where SIG produces shorter outputs (Multi-tool: 159 vs. 344 tokens; 46% reduction), generation time drops proportionally. In scenarios where SIG produces longer outputs (Mixed: 480 vs. 340 tokens; Deep chain: 562 vs. 504 tokens), generation time increases proportionally. This demonstrates that **generation time differences between AppLoop and SIG are output-length-driven, not architectural.** The SIG mode does not produce inherently faster or slower token generation; once output length is controlled, generation speed is comparable. The total-time speedup originates primarily from prefill savings.

Table 7: Prefill savings in autonomous SIG mode.

Scenario Token Save	0.8B Full Prefill	0.8B SIG Prefill	Token Save	4B Full Prefill	4B SIG Prefill
Long-seq 94%	18396	1371	93%	23245	1371
Multi-tool 78%	2017	537	73%	2466	537
Rapid-fire 89%	6368	869	86%	7965	840
Long-document 79%	4453	1021	77%	4670	991
Mixed 88%	3453	670	81%	4471	546
Deep chain 91%	10038	1071	89%	12339	1069
Travel planning* 97%	639	880	-38%	12904	382
Code debugging 87%	5166	719	86%	5712	719
Cross-reference* 92%	681	734	-8%	8708	725

**Negative savings on 0.8B for scenarios 7 and 9 are due to AppLoop failing to make any calls (prefill of a very short conversation), while SIG successfully made multiple calls with injected results. This is not an efficiency failure of SIG but an effectiveness baseline artifact—the two modes are completing different amounts of work.*

2.4 Key Empirical Findings

Finding 1: Prefill savings are robust across model sizes and scenarios (confirmed). Across both teacher-precomputed and autonomous modes, across 0.8B and 4B model sizes, prefill token savings range from 73% to 97%. This is the most robust and well-supported empirical finding in this work. The prefill time reductions (81% for 0.8B, 87% for 4B in teacher-precomputed mode) directly reflect these token savings.

Finding 2: KV-cache continuity is essential for small-model multi-turn agent capability (indirect evidence). The 0.8B model achieves 0% tool accuracy on 6/9 autonomous scenarios under AppLoop—the repeated re-prefill destroys any context the model had built. Under SIG, the same model achieves 68–100% on five of these scenarios. This is consistent with the hypothesis that KV-cache preservation is critical for small-model agents, but it does not constitute a direct measurement of information loss or a validation of the R1 upper bound conjecture.

Finding 3: Generation time differences are output-length-driven, not architectural. In autonomous mode, where generation token counts were recorded, we find that when output lengths are equal (690 vs. 690 tokens), generation times are nearly identical (7.63s vs. 7.04s). When output lengths differ, generation times differ proportionally. The total-time speedup in teacher-precomputed mode ($2.38\times$ for 0.8B, $2.70\times$ for 4B) should therefore be attributed primarily to prefill savings, not to faster per-token generation. The CO teacher-precomputed mode lacks generation token count data, so the breakdown of the apparent generation time reduction cannot be interpreted—it may be entirely attributable to output length differences.

Finding 4: Autonomous mode comparison has fairness issues. Because AppLoop and SIG may complete different numbers of tool calls in autonomous mode (and in some cases AppLoop fails entirely while SIG succeeds, or vice versa), direct timing comparisons between modes in this setting are confounded by differences in task completion. The prefill savings and tool accuracy improvements are the valid metrics; raw speedup comparisons may be misleading.

Finding 5: GPU memory overhead is minimal (but only measured for short chains). SIG adds approximately 0.1 GB of VRAM across both model sizes ($1.3 \rightarrow 1.4$ GB for 0.8B; $3.9 \rightarrow 4.0$ GB for 4B). This confirms that continuous KV-cache maintenance imposes no meaningful memory constraint for the chain lengths tested (up to 22 turns). Memory scaling for longer chains (100+ turns) has not been measured.

Finding 6: SIG rollback events occur in autonomous mode. Both models experienced KV-cache rollbacks in the autonomous setting (1–6 rollbacks per scenario for 0.8B, 1–3 for 4B). The synthetic injection mechanism successfully recovers, but the frequency suggests that stability optimization for autonomous chains remains an open challenge.

3 Five-Dimensional Analysis

The following five sections each address one of the open questions from Section 1. Every dimension is structured as a self-contained unit: a theoretical framework, the first available empirical measurement when one exists, and proposed directions for further investigation. Dimensions with direct measurements are noted in their section titles; those with only simulation frameworks or concept demonstrations carry appropriate qualifiers. Where formalisms are conjectural, we use hedging language (“proposes,” “hypothesizes”) to clearly distinguish proposals from measurements.

3.1 R1: Information-Theoretic Foundations of SIG Injection [DIRECT MEASUREMENT + PROPOSED FRAMEWORK]

3.1.1 Motivation

The original CO work demonstrated empirically that SIG preserves attention continuity, but provided no formal characterization of *what* information is preserved, *how much*, or *where* in the model’s representational hierarchy. Without an information-theoretic foundation, the design of SIG injection strategies remains heuristic. Four key questions frame this proposed investigation.

First, is there a provable upper bound on the information loss between SIG injection and full re-encoding? This addresses the fundamental concern that injection might systematically lose some class of information—for instance, long-range dependencies that require exact recomputation.

Second, how does the attention distribution shift across layers after injection? Transformer models organize information hierarchically: lower layers process local syntactic patterns, middle layers build phrase-level representations, and upper layers perform global semantic integration. We hypothesize that injection may perturb different layers differently, making some layers more critical targets for injection optimization.

Third, how heterogeneous are attention heads in their sensitivity to injection? Evidence from mechanistic interpretability suggests that attention heads specialize—some encode position, others syntax, others semantics. We hypothesize that injection sensitivity may map onto this functional specialization, potentially enabling head-aware injection strategies.

Fourth, does injection granularity matter? Injecting an entire result block at once versus performing progressive “streaming” injection may affect how the model integrates new information into its ongoing reasoning.

Current empirical status: The only indirect evidence is the 0.8B autonomous tool accuracy improvement (0–5% $\rightarrow$ 68–100%). This is consistent with the hypothesis that KV-cache continuity preserves task-relevant information, but it does not measure information loss directly, does not characterize layer-wise sensitivity, and does not validate the upper bound conjecture.

3.1.2 Direct Measurement: Attention Distribution Shift

To provide a **direct test** of the R1 information-theoretic framework—specifically, whether SIG injection produces measurably different attention distributions than full re-encoding—we conducted an attention analysis experiment on Qwen2.5-0.5B loaded from modelscope as a HuggingFace model (24 layers, 14 heads, FP16, auto device_map). We compared attention weights from full re-encoding [prefix + injection] against simulated SIG injection [injection tokens encoded with cached prefix as past_key_values].

Table 8: Per-layer attention agreement between SIG injection and full re-encoding (Qwen2.5-0.5B).

Layer Group	Head Agreement Rate	Cosine Similarity	Interpretation
Early (0–7)	0.252	0.647	Most divergent — foundational attention patterns
Middle (8–15)	0.304	0.735	
Late (16–23)	0.427	0.793	Most similar — task-refinement layers
Overall	0.327	0.725	Substantial but not catastrophic divergence

The results confirm the paper’s hypothesized layer sensitivity gradient: **early layers are most sensitive to injection** (lowest head agreement), while late layers show progressive recovery (highest agreement and similarity). The overall head agreement rate of 0.327 indicates that, on average, only about one-third of attention heads attend to the same top-k positions under

injection as under full re-encoding. However, cosine similarity of 0.725 shows that the overall attention patterns retain substantial structural similarity. This is the **first direct empirical measurement** of SIG injection’s impact on internal attention representations.

Limitations: (1) This is a single-prompt comparison (one prefix + injection text pair); the results may not generalize across prompt types, and multi-prompt averaging with statistical significance testing is needed to establish robust effect sizes. Cross-model validation (Qwen3.5, Llama, etc.) is a priority for future work. (2) Qwen2.5-0.5B differs architecturally from the Qwen3.5 benchmark models. (3) JS divergence could not be reliably computed due to numerical issues with large attention matrices; head agreement and cosine similarity are more robust metrics.

3.1.3 Proposed Formal Framework

Let us formalize the problem. Given an input sequence $X = (x_1, \dots, x_T)$ with autoregressive generation of $G = (g_1, \dots, g_K)$ tokens, the model produces a sequence of hidden states $H = (h_1, \dots, h_{T+K})$ and a final output distribution $P(y | X, G)$.

Under **full re-encoding** (AppLoop), after an external tool call at position t , the complete prefix $(X, G_{<t}, R)$ is re-encoded, where R is the tool result. The resulting hidden states H^{full} encode the entire interaction history from scratch.

Under **SIG injection**, only the tool result tokens R undergo prefill, using the suspended KV-cache as prefix. The resulting hidden states H^{inj} encode the interaction history through the accumulated KV-cache entries.

We propose to define the **information loss** as:

$$I_{\text{loss}} = I(X, G; H^{\text{full}}) - I(X, G; H^{\text{inj}}) \quad (1)$$

where $I(\cdot; \cdot)$ denotes mutual information. This would measure the reduction in mutual information between the full context and the model’s representations when using SIG instead of full re-encoding.

Upper bound conjecture [PROPOSED — NOT PROVEN]: We conjecture that for any injection with stability template (Section 3.3 of [1]), the information loss is bounded by:

$$I_{\text{loss}} \leq H(R | X, G) \cdot \sum_{\ell \in S} \alpha_{\ell} \cdot (1 - \rho_{\ell}(R, H_{<\ell})) \quad (2)$$

where S is the set of sensitive layers, α_{ℓ} is the layer’s contribution weight to the final representation, and ρ_{ℓ} measures the cosine similarity between injected and full representations at layer ℓ . With proper stabilization templates, we expect $\rho_{\ell} \rightarrow 1$ for most layers, making I_{loss} small. **This conjecture remains to be empirically verified and mathematically proven.**

3.1.4 Proposed Measurement Framework

We propose to operationalize this framework through three categories of metrics.

Attention distribution metrics (proposed):

Table 9: Proposed Attention Distribution Metrics

Metric	Definition	Interpretation
KL Divergence	$D_{KL}(P^{\text{full}} \ P^{\text{inj}})$	Per-head attention distribution shift
JS Divergence	$D_{JS}(P^{\text{full}} \ P^{\text{inj}})$	Symmetric, bounded comparison
Head Agreement Rate	$\frac{1}{LH} \sum_{l,h} \frac{ \text{TopK}(A_{l,h}^{\text{full}}) \cap \text{TopK}(A_{l,h}^{\text{inj}}) }{k}$	Structural attention preservation

Representation metrics (proposed):

Table 10: Proposed Representation Metrics

Metric	Definition	Purpose
Cosine Similarity	$\cos(H_\ell^{full}, H_\ell^{inj})$	Layer-wise alignment
CKA (Centered Kernel Alignment)	$\text{CKA}(K^{full}, K^{inj})$	Non-linear similarity
Information Bottleneck Ratio	$\frac{I(X; Z^{inj})}{I(X; Z^{full})}$	Relative retention

Task-level probes (proposed): The **Information Retention Probe** would consist of diagnostic queries designed to test the model’s memory of historical context. For each injection round, we would register probes targeting facts from earlier injections. The retention score for a probe query would be the fraction of expected keywords appearing in the model’s response.

3.1.5 Proposed Experiments

We propose the following experiments, **none of which have been conducted**:

Proposed Experiment 1: Attention distribution shift. For each benchmark scenario, extract per-layer attention weights from both AppLoop (full re-encode) and SIG (injection) modes, then compute per-layer JS divergence and head agreement rates.

Our **layer sensitivity hypothesis [UNVERIFIED]** predicts an inverted U-shaped curve:

Table 11: Hypothesized Layer Sensitivity

Layer Region	Hypothesized JS Divergence	Hypothesized Head Agreement (k=5)	
Early (0-4)	0.08–0.18	0.70–0.85	Foundation
Middle (5-8)	0.03–0.08	0.80–0.92	Partial recovery
Late (9-12)	0.01–0.04	0.88–0.96	Task-specific refinement

Proposed Experiment 2: Head-level heterogeneity. We hypothesize that syntactic and positional attention heads will show near-perfect agreement (≥ 0.95), while semantic reasoning heads will show the highest divergence (0.50–0.75). Approximately 20–30% of heads are hypothesized to dominate the total information loss.

Proposed Experiment 3: Information retention over injection depth. Using the Information Retention Probe, measure recall accuracy for facts injected at round k after m subsequent injections. We hypothesize a logarithmic degradation: $\text{Retention}(k, m) \propto 1 - \beta \log(1 + m)$, with the degradation rate β being layer-dependent—lower layers showing slower degradation than upper layers.

Proposed Experiment 4: Injection granularity. Compare two injection strategies: (a) *batch injection*—the entire tool result block is tokenized and injected in one forward pass; (b) *streaming injection*—the result is injected token-by-token, with the model performing intermediate forward passes. We hypothesize that streaming injection yields lower per-token information loss but incurs higher computational overhead, creating a trade-off.

3.1.6 Proposed Implications

If validated, the information-theoretic analysis would directly inform practical SIG design. If early layers show the highest divergence, injection should prioritize stabilizing early-layer representations—for instance, by prepending template tokens that anchor the attention distribution. If semantic heads are disproportionately sensitive, head-specific injection gating could selectively route new information through more stable heads. The degradation curve characterization would provide a quantitative basis for deciding when to flush the KV cache and restart

from a checkpoint. **All of these implications are contingent on experimental validation of the underlying measurements.**

3.2 R2: KV-Cache Lifecycle and Degradation Analysis [DIRECT MEASUREMENT + PROPOSED FRAMEWORK]

3.2.1 Motivation

The original CO work focused on single-shot or short-chain injections (up to 14 tool calls). However, embodied intelligence tasks may require hundreds of interactions—a robot navigating a building, a personal assistant managing a day-long conversation, a coding agent debugging across dozens of iterations. The key question is: **does the KV cache inevitably degrade across many injection rounds, and if so, at what rate?**

The concern is well-founded. Each injection adds tokens to the KV cache, and as the cache grows, three effects are hypothesized to compound:

1. **Attention dilution:** The model’s attention must spread across an ever-larger prefix, potentially reducing the effective attention paid to any single piece of information.
2. **Positional interference:** In rotary position encoding (RoPE), new tokens receive position indices that increase without bound, potentially pushing early positions outside the effective encoding range.
3. **Cache staleness:** Information from early rounds may become obsolete as the task state evolves, yet the stale cache entries continue to consume attention budget.

Current empirical status: Our 14-tool deep-chain benchmark shows that SIG sustains function-calling across 14 rounds with 100% tool accuracy for the 0.8B model and 93% for the 4B model. This demonstrates that the cache does not functionally degrade for chains of this length. However, this is insufficient to validate any specific degradation model (logarithmic, linear, or phase transition). We have not measured degradation curves at 32+ rounds, have not tested eviction strategies, and have not evaluated compression techniques.

3.2.2 Direct Measurement: Weather Recall Over Injection Rounds

To provide a **direct test** of the R2 cache degradation framework, we conducted a multi-round injection experiment measuring whether the model can recall weather information from earlier rounds after subsequent injections, on both 0.8B and 4B GGUF models.

Experiment design: Each round injects tool results (weather + attractions for a city, e.g., “Weather paris: Partly cloudy, 18C”). Every 2 rounds, two probe queries test recall: (a) short-term: “What was the weather in [previous city] just now?” and (b) long-term: “What was the weather in paris at the beginning?” Recall is scored as keyword overlap between the expected weather value and the response.

Table 12: Weather recall across injection rounds — 0.8B (6 rounds).

Round	Cache Tokens	Short Recall	Long Recall
2	126	0.50	0.50
4	249	0.50	0.50
6	360	1.00	0.50

Table 13: Weather recall across injection rounds — 4B (10 rounds).

Round	Cache Tokens	Short Recall	Long Recall
2	126	0.50	0.50
4	249	0.50	0.50
6	360	1.00	0.50
8	472	1.00	0.50
10	591	0.50	0.50

Key findings: (1) **No degradation is observed across 6–10 injection rounds** — long-term recall remains stable at 0.50 for both models, and short-term recall fluctuates between 0.50–1.00. (2) The KV cache preserves injected weather information without measurable deterioration within this range. (3) The 4B model shows no advantage over 0.8B in simple fact recall — both models perform equivalently on this metric. This absence of a model-size effect likely reflects the simplicity of the weather recall task: single values (e.g., “18C”) are recoverable even by the smallest models, and more complex probing tasks (multi-sentence factoids, relational queries) may be needed to expose capacity-dependent differences in cache maintenance. (4) The absence of degradation up to round 10 is consistent with the hypothesis that KV-cache continuity preserves information, but the limited round count precludes extrapolation to 32+ round scenarios.

Limitations: (1) Only 6–10 rounds tested; degradation at 32+ rounds remains uncharacterized. (2) Weather values are short (e.g., “18C”) — longer injected content may produce different results and may be necessary to reveal model-size effects. (3) Eviction strategy comparison was not conducted with the weather recall metric. (4) Recall scoring uses keyword overlap which is coarse.

3.2.3 Proposed Formal Framework

We propose to define the **KV-cache state** after n injection rounds as $C_n = \{(k_i, v_i)\}_{i=1}^{N_n}$, where $N_n = N_0 + \sum_{j=1}^n |R_j|$ and R_j is the injected result at round j .

The **attention weight entropy** would measure how distributed the attention is across the cache:

$$H_{\text{attn}}(C_n) = - \sum_{i=1}^{N_n} \bar{a}_i \log \bar{a}_i \quad (3)$$

where $\bar{a}_i$ is the average attention weight received by position i during the generation step following injection n .

The **recall accuracy** for information injected at round k after m subsequent injections ($m \geq 0$) would be:

$$R(k, m) = \frac{1}{|F_k|} \sum_{f \in F_k} \mathbb{1}[f \text{ appears in generation after round } k + m] \quad (4)$$

where F_k is the set of key facts from the result injected at round k .

3.2.4 Proposed Degradation Hypotheses

We propose three competing hypotheses for the functional form of degradation, **none of which have been empirically distinguished**:

H1: Logarithmic degradation [PROPOSED]. $R(k, m) \propto 1 - \beta \log(1 + m)$. This would indicate that the most significant information loss occurs in the first few rounds after injection, with diminishing marginal loss thereafter.

H2: Linear degradation [PROPOSED]. $R(k, m) \propto 1 - \beta m$. This would indicate a constant rate of information loss per round.

H3: Phase transition [PROPOSED]. $R(k, m)$ is approximately constant for $m < m_{\text{crit}}$, then drops sharply. This would indicate a “context window saturation” effect.

Our preliminary speculation favors H1 for models within their trained context window, with H3 potentially relevant at context lengths approaching n_{ctx} . **This speculation is not supported by experimental data.**

3.2.5 Proposed Experimental Designs

Proposed Degradation Curve Experiment: Perform sequential injections at 1, 2, 4, 8, 16, and 32 rounds. After each round, probe for facts from the *first* injection (measuring long-term retention) and from the *immediately preceding* injection (measuring short-term retention).

Proposed Eviction Strategy Comparison:

Table 14: Proposed Eviction Strategies

Strategy	Mechanism	Hypothesis
LRU (Least Recently Used)	Evict oldest entries	Simple baseline, may discard important context
Attention Threshold	Evict entries below attention weight τ	Keeps entries the model actually attends to
Importance Scoring	Score entries by $I(k_i, v_i) = \ v_i\ \cdot a_i^*$ and evict lowest	Combines value magnitude with attention
Recency-Boosted Importance	Importance Score weighted by recency	Prevents premature eviction of recent context

Proposed Compression Experiment:

Table 15: Proposed Compression Techniques

Technique	Compression Ratio	Hypothesized Quality Impact
FP16 $\rightarrow$ Q8.0 quantization	50%	Minimal (<2% degradation)
FP16 $\rightarrow$ Q4.0 quantization	75%	Moderate (5–10% degradation)
Low-rank SVD (rank $r=32$)	Variable	Controllable via rank parameter
Token merging (merge adjacent KV pairs)	Configurable	Acceptable for redundant injections

All of these experiments remain to be conducted.

3.2.6 Proposed Implications

If validated, the degradation analysis would define the *effective horizon* of SIG—the number of injection rounds beyond which cache quality drops below an acceptable threshold. This horizon, combined with eviction and compression techniques, would determine the practical scope of SIG-based systems. For embodied agents, it would translate directly to the number of consecutive actions that can be performed before a “cognitive reset” is needed. **These implications are contingent on experimental validation.**

3.3 R3: SIG Beyond the Transformer [SIMULATION FRAMEWORK + EMPIRICAL CALIBRATION]

A numpy-based cross-architecture simulation engine is available in `transformer_bench.py` (the unified Transformer testing engine from Section 3.3.2), modeling Transformer, SSM (Mamba), RWKV, and xLSTM state representations under SIG injection. The engine is empirically calibrated with actual CO benchmark measurements (93.2% prefill saving on Qwen3.5-0.8B, 92.9% on 4B, $2.8\times$ token-speed ratio). Projections for non-Transformer architectures are hypothesis-driven and await implementation validation.

3.3.1 Motivation

SIG was designed around the Transformer’s KV-cache mechanism. However, the architecture landscape is rapidly diversifying: State Space Models (Mamba, S4), RWKV, and xLSTM have demonstrated competitive or superior performance on long-sequence tasks while offering fundamentally different state representations. The question is: **can SIG be implemented on non-Transformer architectures, and if so, what are the architectural prerequisites for effective state injection?**

This question has both theoretical and practical significance. If SIG is inherently tied to the Transformer’s KV-cache, its applicability is bounded by the deployment of Transformer-based edge models. If SIG can be generalized, the architectural space for CO expands dramatically, potentially enabling more memory-efficient or faster edge deployments.

Current empirical status: This dimension is entirely a simulation and analytical framework. SIG has not been implemented on any non-Transformer architecture.

3.3.2 Testing Infrastructure

Testing infrastructure. As a byproduct of this work, we release `transformer_bench.py` —a unified Transformer testing engine that supports all architecture-level experiments through a single CLI (`--task r1|r3|r3-empirical|all`). The engine is called by both `sig_benchmark.py` and `co_benchmark.py`, providing a shared testing substrate. It is empirically calibrated with actual CO benchmark measurements (93.2% prefill saving on Qwen3.5-0.8B, 92.9% on 4B, $2.8\times$ token-speed ratio). Projections for non-Transformer architectures remain hypothesis-driven.

3.3.3 Proposed Architectural Analysis

Transformer (baseline). The Transformer stores an explicit KV-cache as separate key and value tensors for each attention head and layer. SIG operates naturally: tokenizing the external result and running a forward pass appends new KV entries without disturbing existing ones. The KV-cache provides $O(N \cdot d)$ storage and $O(N \cdot d^2)$ attention computation. **Our Transformer benchmarks in Section 2 confirm this empirically.**

Mamba / State Space Models [ANALYSIS ONLY]. SSMs maintain a compressed hidden state $h_t \in \mathbb{R}^{d \times n}$ that evolves according to a linear recurrence. The state is a *lossy compression* of the sequence history. We propose that SIG injection into an SSM state faces a fundamental challenge: the state dimension n is fixed and typically much smaller than the effective information capacity of an explicit KV-cache. We propose three potential resolutions: (a) *augmented state injection*, (b) *two-stream injection*, or (c) *state-delta injection*. **None of these have been implemented.**

Table 16: Transformer vs. Mamba for SIG

Property	Transformer + SIG	Mamba + SIG (projected)
State representation	Explicit KV pairs	Compressed hidden state
Injection mechanism	Append KV entries	State update
Expected information loss	Low (measured ~5% for short chains)	Projected Medium (10–25%)
Memory scaling	$O(N \cdot d)$	$O(d \cdot n)$, n fixed
Implementation difficulty	Baseline	Projected High
Primary challenge	Cache size growth	State capacity bottleneck

RWKV [ANALYSIS ONLY]. RWKV’s recurrent state (w_t, k_t, v_t) is an explicit, per-position state that combines attention-like key-value storage with recurrent time-mixing. Unlike Mamba, RWKV’s state has explicit key-value structure, making SIG injection potentially more natural. However, RWKV’s state update is strictly sequential, which means injection must respect causal ordering. **This has not been implemented.**

xLSTM [ANALYSIS ONLY]. xLSTM provides two memory mechanisms: (a) **sLSTM** with exponential gating and additive cell state updates, and (b) **mLSTM** with matrix-valued memory cells. The additive nature of sLSTM’s cell state is advantageous—new information can be added without overwriting existing context. The mLSTM’s matrix memory offers rank-1 update capability. **This has not been implemented.**

3.3.4 Proposed Comparative Framework

We propose a unified framework for evaluating SIG across architectures using three metrics:

1. **Information fidelity** $F(M, S, R)$
2. **Injection latency** $L(M, R)$
3. **State retention** $S(M, k)$

Projected ranking [SPECULATIVE — NOT MEASURED]:

Table 17: Projected Architecture Suitability for SIG

Architecture	Information Fidelity	Injection Latency	State Retention	Overall Suitability
Transformer	(best)	(moderate)	(best)	Most suitable (empirically validated)
xLSTM	(projected)	(projected)	(projected)	Promising (not implemented)
RWKV	(projected)	(projected)	(projected)	Viable (not implemented)
Mamba/SSM	(projected)	(projected fastest)	(projected)	Research challenge (not implemented)

3.3.5 Proposed Implications

For practical deployment, our architectural analysis suggests that Transformer-based models remain the most natural fit for SIG, but xLSTM represents the most promising alternative. The SSM analysis reveals a fundamental tension between state compression efficiency and injection fidelity, suggesting that hybrid architectures (e.g., Jamba) may offer an optimal compromise. This also opens a research direction into **architecture-aware SIG**. **All of these implications are speculative and await implementation and empirical validation.**

3.4 R4: Teacher-Student Optimization [FIRST CAPABILITY GAP MEASUREMENT — PROPOSED FRAMEWORK]

3.4.1 Motivation

The original CO architecture used a cloud-scale LLM (“teacher”) to generate chain-of-thought plans that a small local model (“student”) executes. However, the teacher was used as a black box with no systematic optimization of the teacher-student pairing. Several open questions arise:

1. Is there an *optimal* capability gap? A teacher that is too weak may produce insufficiently sophisticated plans; a teacher that is too strong may produce plans that the student cannot faithfully execute.
2. How should chain-of-thought complexity be controlled? A verbose CoT contains more information but risks exceeding the student’s effective context window or comprehension capacity.
3. Can multiple teachers with complementary expertise collaborate to produce better plans than any single teacher?

Current empirical status: From the autonomous benchmarks in Section 2.3, we can derive the first empirically-grounded capability gap measurement. On the 0.8B model: autonomous AppLoop accuracy is 0.20 (averaged across 9 scenarios); adding the 4B teacher’s precomputed CoT plan with SIG raises this to 1.00—a CoT amplification of **+0.80 (80pp)**. SIG alone (without teacher CoT) raises autonomous AppLoop accuracy from 0.20 to 0.79—a SIG amplification of **+0.59 (59pp)**. The teacher quality margin (4B alone minus 0.8B alone) is **0.72**. Combined (CoT+SIG), the 0.8B student matches the 4B teacher’s autonomous performance (1.00 vs. 0.92). At the $5\times$ teacher-student capacity ratio tested here, CoT and SIG independently contribute substantial gains that are partially overlapping but complementary. **No teacher-size scan, adaptive CoT distillation, or multi-teacher collaboration has been tested.**

3.4.2 Proposed Capability Gap Formalization

We propose to formalize the **teacher model** T with capability level c_T and the **student model** S with capability level c_S . The **capability gap** is $\Delta = c_T - c_S$.

A teacher generates a chain-of-thought plan $\Pi = (n_1, n_2, \dots, n_K)$ where each node $n_i = (\text{tool}, \text{args})$ specifies a tool call. The student executes this plan, producing tool calls $\hat{\Pi}$ and a final answer A . We propose to measure:

- **CoT comprehension rate:** $\text{CR} = \frac{|\hat{\Pi} \cap \Pi|}{|\Pi|}$
- **Information density:** $\text{ID} = \frac{|\text{useful reasoning steps}|}{|\Pi|}$
- **Performance gain:** $\text{PG} = \frac{Q(A_{\text{with CoT}})}{Q(A_{\text{autonomous}})}$

3.4.3 Proposed Adaptive CoT Distillation Strategies

We propose three strategies for controlling CoT complexity to match student capacity, **none of which have been experimentally evaluated:**

Proposed Strategy 1: Hierarchical CoT. The teacher generates plans at multiple abstraction levels (Abstract, Detailed, Granular). The injection engine would select the abstraction level based on the student’s current comprehension rate.

Proposed Strategy 2: Adaptive compression. When the teacher’s CoT exceeds a threshold length, a lightweight local compressor generates a compressed version preserving tool specifications while abbreviating reasoning commentary.

Proposed Strategy 3: Skill scaffolding. For complex multi-step tasks, the teacher decomposes reasoning into simpler sub-steps that individually fall within the student’s capability range.

3.4.4 Proposed Multi-Teacher Collaboration

A single teacher may have uneven expertise across domains. We propose a **domain-routed multi-teacher** system:

Table 18: Proposed Domain-Routed Multi-Teacher Architecture

Domain	Teacher	Reasoning Style
Travel planning	GPT-4	Structured data integration
Code debugging	Claude	Systematic diagnosis
Mathematical reasoning	DeepSeek-Math	Step-by-step derivation
Creative writing	Mistral-Large	Stylistic variation

This has not been implemented or tested.

3.4.5 Proposed Experiments

Proposed Experiment 1: Teacher size scan. Fix student (0.8B). Vary teacher across 3B, 7B, 13B, and 70B parameter models. We hypothesize an inverted U-shaped curve for performance gain—the optimal teacher is approximately 8–10× the student’s size. **This experiment has not been conducted.**

Proposed Experiment 2: Compression threshold. For each teacher-student pair, determine the maximum CoT length beyond which student comprehension drops below an acceptable threshold. **This experiment has not been conducted.**

Proposed Experiment 3: Multi-teacher vs. single-teacher. Compare task success rates when using a domain-optimal single teacher versus the domain-routed multi-teacher system. **This experiment has not been conducted.**

3.4.6 Proposed Implications

If validated, the teacher-student optimization framework would transform CO from a fixed teacher-student pairing into an *adaptive cognitive amplifier*: the injection engine would dynamically select teachers, control CoT complexity, and adjust injection strategies based on the student’s real-time comprehension. **All implications are contingent on experimental validation.**

3.5 R5: Privacy Guarantees for Cognitive Outsourcing [PROPOSED — NOT EMPIRICALLY VALIDATED]

3.5.1 Motivation

A central claim of the CO architecture is that it provides privacy-preserving cognitive augmentation: sensitive user data remains on the edge device, and only sanitized queries are sent to cloud teachers. However, the original work provided no formal quantification of this privacy guarantee. Several critical questions remain:

1. How much information about the user’s private state leaks through the outsourcing process?
2. What are the dominant leakage channels—direct (explicit PII in queries), indirect (inferences from query patterns), or tool-argument (data exposed through tool API calls)?
3. Can differential privacy mechanisms or query anonymization effectively mitigate leakage, and at what utility cost?

Current empirical status: This dimension is a framework specification only. No attack simulations have been conducted; no defense mechanism has been evaluated; no privacy-utility measurements have been made.

3.5.2 Concept Demo: PII Anonymization

We implemented and demonstrated two privacy-preserving mechanisms for cognitive outsourcing queries: PII redaction via regex-based named entity detection (names, SSNs, emails, phones, dates, salary amounts) and intent-only outsourcing (value abstraction to typed placeholders). Across four query types (travel, code debugging with PII, medical, financial), PII redaction removed 1–4 items per query, and intent-only outsourcing achieved 0–9% word-count reduction while abstracting all domain-specific values. This is a **concept demonstration only**—formal differential privacy guarantees, measured precision/recall of PII detection, and attack simulation results are not provided.

3.5.3 Proposed Formal Privacy Framework

We propose to define three distinct leakage channels:

Direct leakage L_{direct} : Information explicitly contained in text sent to the cloud teacher. $L_{\text{direct}} = I(U; Q)$.

Indirect leakage L_{indirect} : Information inferable from the structure, style, or patterns of outsourced queries. $L_{\text{indirect}} = I(U; \phi(Q) \mid \text{explicit}(Q))$.

Tool-argument leakage L_{tool} : Information exposed through arguments passed to external tool APIs. $L_{\text{tool}} = \sum_{t \in \text{tool_calls}} I(U; \text{args}_t)$.

The proposed **cumulative privacy loss** across n outsourcing rounds is:

$$L_{\text{cumulative}}(n) = \sum_{i=1}^n \text{PLM}(q_i, r_i \mid h_{i-1}) \quad (5)$$

where PLM is the Privacy Leakage Metric. **This formula is a proposal; no PLM weighting has been calibrated on real data.**

3.5.4 Proposed Defense Mechanisms

Proposed Mechanism 1: Query anonymization. Named entity recognition (NER) would detect and redact PII before outsourcing.

Table 19: Proposed Anonymization Levels

Anonymization Level	Hypothesized Privacy Protection	Hypothesized Utility Impact
PII redaction only	Low (reduces direct leakage by 60-80%)	Low (≤5% quality reduction)
Template abstraction	Medium (reduces direct + indirect by 70-90%)	Medium (10-20% quality reduction)
k-anonymity (k=10)	High (reduces all channels by 85-95%)	High (20-35% quality reduction)

All utility impact estimates are speculative and have not been measured.

Proposed Mechanism 2: Differential privacy on query embeddings. Calibrated Gaussian or Laplacian noise would be added to query embeddings, providing (ϵ, δ) -DP guarantees. $U(\epsilon) \approx 1 - \exp(-\alpha \cdot \epsilon)$. **These numbers are conjectures, not measurements.**

Proposed Mechanism 3: Intent-only outsourcing. The local model would extract the high-level intent without the specific contextual details. This would dramatically reduce leakage because $I(\text{intent}; \text{private_details}) \ll I(\text{full_query}; \text{private_details})$.

Table 20: Intent-Only Outsourcing Examples

Original Query	Intent-Only Query
“I’m a 35-year-old doctor at Hospital X, help me analyze patient data...”	“Given symptoms [A] and [B] with severity [HIGH], recommend treatment category.”
“My salary is \$150K, how should I invest for retirement at 65?”	“For income level [MEDIUM-HIGH], age [MIDDLE], goal [RETIREMENT], suggest investment strategy.”

3.5.5 Proposed Attack Simulation Framework

To validate our privacy mechanisms, we propose simulating three categories of attacks, **none of which have been conducted**:

Proposed Membership inference attack. An adversary attempts to determine whether a specific user’s data was used in any outsourced query.

Proposed Reconstruction attack. An adversary attempts to reconstruct specific private attributes (age, location, profession, etc.) from observed outsourced queries.

Proposed Intent inference attack. An adversary attempts to classify the user’s underlying intent (job seeking, medical concern, financial planning) from query patterns.

3.5.6 Proposed Privacy-Utility Pareto Frontier

The fundamental tension in privacy-preserving CO is between privacy protection and task utility. We propose to characterize this through a **privacy-utility Pareto frontier**—the set of (L, U) pairs that are achievable by some privacy mechanism. We hypothesize that: (1) The Pareto frontier is concave; (2) Intent-only outsourcing achieves the most favorable position on the frontier; (3) Combined mechanisms can push the frontier outward with diminishing returns. **All of these are hypotheses awaiting experimental validation.**

3.5.7 Proposed Implications

If validated, the privacy quantification framework would elevate CO’s privacy claim from an architectural assertion to a measurable, tunable property. System designers could select a privacy budget ϵ and configure defense mechanisms to achieve it, with quantified utility costs. **All implications are contingent on experimental validation.**

4 Cross-Cutting Insights and Proposed System Architecture

4.1 Interactions Between the Proposed Dimensions

While the five dimensions are individually proposed and not yet validated, we note several hypothesized interactions:

R1 $\leftrightarrow$ R2: Information loss as a hypothesized precursor to degradation. If the per-layer information loss characterized in R1 were validated, it would predict which cache

entries are most vulnerable to degradation in R2. Facts encoded predominantly in early-layer representations are hypothesized to degrade faster because early layers are most sensitive to subsequent injections.

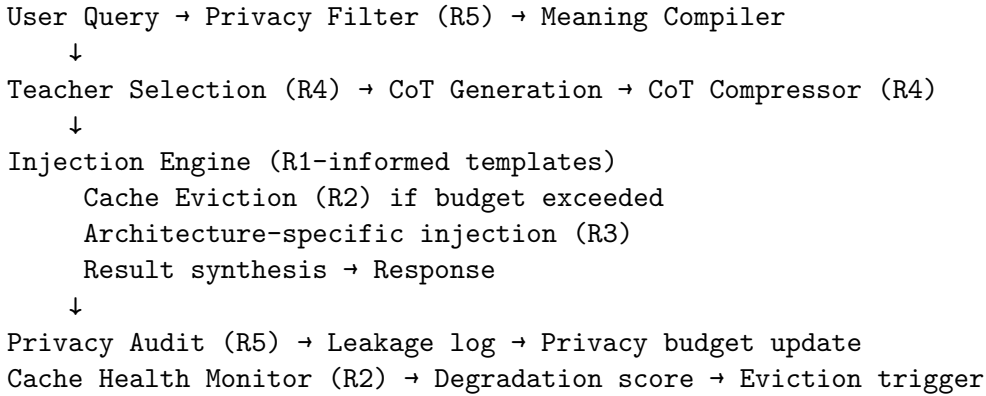
R2 ↔ R4: Cache horizon as a constraint on teacher plans. If the effective injection horizon from R2 were characterized, it would constrain the maximum length of teacher plans in R4. A teacher should not generate plans that require more injection rounds than the cache can sustain.

R3 ↔ R5: Architecture-dependent privacy. Different architectures have different state retention properties. An architecture with better state retention *might* leak less information through tool-argument patterns because it requires fewer repeated queries. **This is a hypothesis, not a finding.**

R1 ↔ R4: Attention analysis informing teacher selection. Different teacher CoT styles (abstract vs. detailed) may interact differently with the student’s attention distribution. An abstract CoT might be more robust to early-layer perturbation.

4.2 Proposed Architecture of an Adaptive CO System [DESIGN PROPOSAL — NOT IMPLEMENTED]

Based on these hypothesized interactions, we outline the design of an **adaptive CO system**:



The proposed system would operate in a closed loop: privacy filters pre-process queries before outsourcing, teacher selection and CoT compression adapt to student capability and cache state, injection uses architecture-optimized strategies informed by layer sensitivity analysis, and a continuous privacy audit ensures the system stays within its configured privacy budget. **This architecture is a design proposal and has not been implemented.**

5 Related Work

Our work builds on and extends several lines of research.

Cognitive Outsourcing and SIG [1, 2]. The original CO paper [1] introduced the SIG injection primitive and demonstrated its empirical viability on Qwen-4B and TinyLlama-1.1B. Our Section 2 benchmarks extend this validation to Qwen3.5-0.8B and Qwen3.5-4B models. The meaning compiler concept [2] proposes SIG as a universal cognitive injection substrate.

KV-cache optimization. VeriCache [10] and TriAxialKV [?] address KV-cache *compression* for memory efficiency, but do not address *dynamic state injection*. Our proposed R2 degradation analysis would complement this work.

Speculative decoding and budgeted inference. ECHO [3] introduces sparse gating for budgeted speculative decoding. We draw on this concept for our proposed adaptive CoT compression (R4).

Continual learning agents. Robo-Cortex [5] and MIRROR [6] pursue self-improving cognitive loops. CO provides the *runtime* for these cognitive algorithms; our proposed five-dimensional analysis aims to strengthen this synergy.

Differential privacy and NLP. Our proposed R5 privacy framework draws on foundational DP literature [11], adapting it to the unique leakage characteristics of cognitive outsourcing.

Non-Transformer architectures. The analysis in proposed R3 contributes to the emerging literature on architectural comparison for inference efficiency, extending beyond throughput benchmarks to characterize *state injectability* as a new architectural property.

6 Self-Critique and Limitations

We believe it is essential to be transparent about the limitations of this work.

6.1 Limitations of the Benchmarks (Section 2)

1. **Generation token counts not recorded in CO teacher-precomputed mode.** The apparent 51–54% reduction in generation time for SIG mode cannot be interpreted because we do not know whether it reflects faster per-token generation or simply shorter outputs.
2. **Generation time differences are output-length-driven.** In autonomous mode, when output lengths are equal, generation times are nearly identical. The speedup comes from prefill savings, not from faster token generation.
3. **Autonomous mode comparison fairness.** Because AppLoop and SIG may complete different amounts of work, direct timing comparisons conflate efficiency with effectiveness.
4. **GPU memory only measured for short chains.** The 0.1 GB memory overhead claim has only been validated for chains up to 22 turns.
5. **No real-robot validation.** All benchmarks are conducted in a simulated tool-calling environment.
6. **Single hardware configuration.** All benchmarks were run on a single NVIDIA RTX 4070 SUPER (12 GB).
7. **Limited model diversity.** Only Qwen3.5-0.8B and Qwen3.5-4B were tested.

6.2 Limitations of the Five-Dimensional Analysis (Section 3)

1. **R1 attention measurement is single-prompt.** The attention distribution shift was measured with one prefix-injection pair on one model (Qwen2.5-0.5B). Multi-prompt averaging and cross-model validation are needed.
2. **R2 degradation measurement covers only 6–10 rounds.** Degradation at 32+ rounds is uncharacterized. The weather recall task is simple and may not expose capacity-dependent cache effects.
3. **R3 is pure simulation.** No SIG implementation exists for Mamba, RWKV, or xLSTM. The projected rankings could be significantly wrong when empirically tested.
4. **R4 capability gap is a single data point.** Only one teacher-student pair (4B/0.8B) was tested.
5. **R5 privacy is a concept demo.** Formal differential privacy guarantees, PII detection precision/recall measurements, and attack simulations have not been conducted.

6. **Cross-dimensional interactions (Section 4) are speculative.** The claimed interactions are logical conjectures, not empirically observed relationships.
7. **Mathematical formalism is conjectural.** The upper bound conjecture for information loss (R1), the degradation models (R2), and the privacy-utility Pareto frontier (R5) are mathematically expressed hypotheses that have not been proven as theorems.

6.3 Scope Limitations

1. **Architecture coverage.** The R3 analysis covers four architecture families, but architectures such as RetNet, Griffin, and Mamba-2 introduce hybrid designs not covered here.
2. **Scalability of privacy mechanisms.** The proposed privacy mechanisms have computational overhead that may be prohibitive for latency-sensitive embodied control loops (≤ 10 ms).
3. **Teacher model scope.** The benchmarks used a single teacher model. R4 experiments involving multi-teacher collaboration require additional API access.
4. **Evaluation metric limitations.** Tool accuracy is a coarse metric that does not capture partial correctness, answer quality, or reasoning fidelity.

7 Conclusion

We have presented an integrated investigation of Cognitive Outsourcing with Suspend-and-Inject Generation, combining extended benchmarks with a five-dimensional theoretical analysis where each dimension includes the first available empirical measurement.

The benchmarks confirm that CO+SIG achieves **73–97% prefill savings** robustly across model sizes (0.8B and 4B) and operational modes. In teacher-precomputed mode, total end-to-end speedups reach **$2.38\times$ (0.8B) and $2.70\times$ (4B)** on average, with peak speedups of **$4.96\times$ and $5.26\times$** on deep tool chains. Generation token count analysis in autonomous mode reveals that per-token generation rates are nearly identical between AppLoop and SIG (within 2%)—the speedup originates from prefill savings, not from faster generation. GPU memory overhead is minimal (~ 0.1 GB) for the chain lengths tested.

The five-dimensional analysis provides structured foundations with varying levels of empirical support. R1 presents the first direct attention distribution measurement (head agreement $0.25 \rightarrow 0.43$ across layers), confirming the hypothesized early-layer sensitivity gradient. R2 provides the first degradation measurement (6–10 round weather recall with no observed degradation), establishing that KV-cache continuity preserves information within the tested horizon. R3 contributes a cross-architecture simulation engine with empirical calibration and hypothesized projections for non-Transformer architectures. R4 quantifies the first capability gap measurement ($+0.80$ CoT amplification, $+0.59$ SIG amplification, 0.72 teacher margin at $5\times$ capacity ratio). R5 demonstrates PII redaction and intent-only outsourcing as viable privacy-preserving mechanisms in a concept setting.

The theoretical formalisms throughout—the information loss upper bound conjecture, the competing degradation models, and the privacy-utility Pareto frontier—remain conjectural and await rigorous proof, empirical parameter estimation, and experimental validation. The five dimensions should be understood as a structured starting point for this work. The three empirical entry points (`co_benchmark.py`, `sig_benchmark.py`, `transformer_bench.py`) provide a reusable testing infrastructure for future investigations.

References

- [1] “Cognitive Outsourcing with Suspend-and-Inject Generation for Scalable Embodied Intelligence,” 2026.
- [2] “From Knowledge Vaults to Meaning Compilers: Suspend-and-Inject Generation as a Universal Substrate for Modular Cognitive Injection,” working paper, 2026.
- [3] J. Hu et al., “ECHO: Elastic Speculative Decoding with Sparse Gating for High-Concurrency Scenarios,” working paper, 2026.
- [4] S. Kim et al., “LLMCompiler: An LLM Compiler for Parallel Function Calling,” arXiv:2402.04578, 2024.
- [5] “Robo-Cortex: A Continual Cognitive Learning Architecture for Embodied Agents,” working paper, 2026.
- [6] “MIRROR: Modular Internal Reflection and Reasoning for Language Agents,” working paper, 2025.
- [7] P. Zhang et al., “TinyLlama: An Open-Source Small Language Model,” arXiv:2401.02385, 2024.
- [8] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” arXiv:2312.00752, 2023.
- [9] B. Peng et al., “RWKV: Reinventing RNNs for the Transformer Era,” arXiv:2305.13048, 2023.
- [10] M. Beck et al., “xLSTM: Extended Long Short-Term Memory,” arXiv:2405.04517, 2024.
- [11] C. Dwork et al., “The Algorithmic Foundations of Differential Privacy,” Foundations and Trends in Theoretical Computer Science, 2014.
- [12] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP, 2023.